

Cryptography, trusted third parties and escrow

S J D Phoenix

The promise of tomorrow's electronic business environment will not be fully realised unless government, business and consumers have a high level of confidence in the security of that environment. The use of cryptography can provide this confidence and facilitate the establishment of a global electronic market-place. Cryptographic techniques offer a range of services including confidentiality, authenticity, integrity and anonymity. We examine some of these techniques and explain why a trusted third party infrastructure is seen as a crucial component in the successful introduction of an electronic business arena. The widespread use of cryptography, however, also presents possible new opportunities for criminal activity. To mitigate against this many governments have proposed the provision of escrowed cryptographic services — we review a particular scheme favoured by the UK government.

1. Introduction

The establishment of a global electronic business environment is one of the most exciting challenges facing the telecommunications industry today. Confidence in that environment, or the lack of it, will be a crucial factor deciding its success or failure. Possibly the most important techniques available to us to generate that confidence fall within the purview of cryptography. Cryptography aims to achieve two essential, but independent, goals. The first of these goals is to ensure the confidentiality of data on an open network. The second is to ensure the authenticity of that data. With these techniques and the infrastructure to support their use in place we can envisage a safe, secure open electronic environment for conducting business.

The purpose of this paper is to review the basic features and practice of cryptography. We shall begin with an overview of the basic techniques and concepts. In particular the important concept of a digital signature will be introduced and its significance for an electronic commerce framework explained. This leads us naturally on to the notion of certification and trusted intermediaries. We shall conclude with a discussion of the political issues surrounding the use of strong cryptographic techniques and their importance for BT.

2. Basic cryptography

2.1 Confidentiality and authenticity

Historically, the protection of confidentiality has been the most important function of a cryptography system. This is partly because until relatively recently cryptography has been the exclusive remit of those government

organisations to whom secrecy is paramount. The needs of today's business community go beyond the provision of simple confidentiality. Authenticity is at least as important if not more so. If the trend to a global electronic market-place continues, more and more of our transactions will need to be protected by strong cryptographic techniques.

For so-called classical or secret-key cryptographic systems, confidentiality is achieved by taking the data to be protected and, using some secret information, transforming it into a form unintelligible to anyone who does not possess this secret piece of information. This extra secret piece of information is the key. A principal difficulty in the operation of secret-key cryptosystems is that of distributing the secret key to legitimate users. The obvious analogy is that of locking a box with a message inside it. Even though we may know how the lock works it is difficult to unlock without the key. Anyone who wants to read the data in the box needs a copy of the secret key.

Cryptography achieves the same kind of thing using mathematical transformations. We can think of the mathematical algorithm which scrambles the data as the locking mechanism and this extra information, the key, which is usually a sequence of binary digits, as the device to activate or de-activate the algorithm. Cryptography systems should be designed on the assumption that an attacker knows the details of the algorithm so that the confidentiality of a message is not dependent in any way upon keeping the algorithm secret.

One of the most celebrated algorithms for encryption is known as the one-time pad. This cipher provides perfect

secrecy and cannot be broken, if properly implemented. The algorithm is extremely simple. The ciphertext is produced from the bit-wise exclusive-or of the plaintext with the key. The decryption algorithm reproduces the plaintext by taking the bit-wise exclusive-or of the ciphertext with the key. The main problem with this algorithm is that to achieve perfect secrecy the key must be random, as long as the message is to be encrypted and used only once. Modern secret-key systems aim to achieve a high degree of secrecy with a much shorter and more convenient key.

There is another kind of cryptography known as public-key cryptography which achieves confidentiality by a different route. To resort to the analogy of a lockable box, public-key cryptography is like having a box that everyone can lock but the unlocking key is kept secret. The mathematical transformations that scramble the data for public-key systems are of a very different nature to those used for secret-key cryptography. At first glance it might be thought that public-key cryptography solves the problem of having to get secret keys to each user of a cryptosystem. However, because the locking keys are widely distributed the authenticity of these keys must be guaranteed in some way. The difficulty in public key systems shifts from that of distributing a secret key to one of authenticating a public key.

Authenticity is a more subtle concept than confidentiality and is also more difficult to achieve. Secrecy and authenticity are independent attributes of a cryptographic system and one does not necessarily imply the other. An interesting classification of the difference between authenticity and secrecy has been formulated by Xuejia Lai:

- a technique provides secrecy if its purpose is to determine who can receive a message,
- a technique provides authenticity if its purpose is to determine who can send a message.

Message authenticity is therefore concerned with the integrity of the message and its sender. As we shall see cryptographic techniques can be used to provide very strong mechanisms for ensuring that integrity.

2.2 Elements of a cryptosystem

The basic elements of any cryptosystem are depicted in Fig 1. Our protagonists, Alice and Bob, wish to communicate in secret and their adversary is Eve, the eavesdropper, who wishes to discover the content of their exchange. The message, m (also known as the plaintext), is fed into an encryption algorithm together with a key, k , to produce the cryptogram or ciphertext, c . To decrypt the ciphertext and recover the message it is necessary to feed the decryption key, d into the decryption algorithm together

with the ciphertext. A cryptosystem whose encryption and decryption keys are simply related in the sense that knowledge of one gives knowledge of the other is known as a symmetric-key or secret-key system. A system for which the encryption and decryption keys are different and one is not easily calculated from the other is known as an asymmetric-key or public-key system. We shall return to public-key systems later in the paper, but for the meantime we shall concentrate on secret-key systems. This allows a minor clarification of the discussion but is not significant.

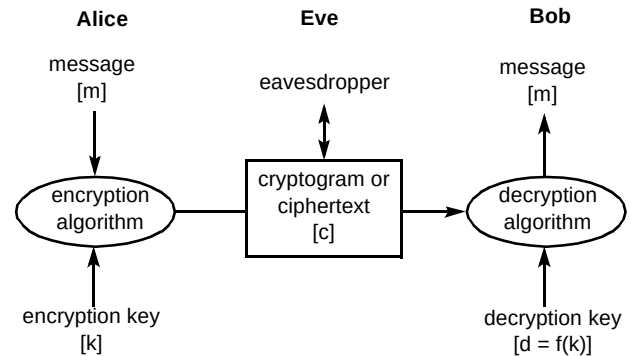


Fig 1 The basic elements of a cryptosystem. Alice encrypts the message using an encryption key and sends it to Bob who decrypts it using his decryption key. It is assumed that the eavesdropper has access to the ciphertext but not the keys.

It is a basic assumption when designing any cryptosystem that an attacker knows the details of the encryption and decryption algorithms. This is certainly true of many algorithms in current use such as DES (the data encryption standard) and IDEA (international data encryption algorithm). However, some organisations who supply products utilising cryptography do not reveal the algorithms used. The argument for this approach is summarised by the question: 'Why should we make it easier for an attacker?' This approach is often criticised on the grounds that secret algorithms have not withstood the scrutiny of the international community and their strength cannot therefore be independently verified. DES, for example has been known for over 20 years and a successful feasible attack has still not been demonstrated despite intensive efforts. Both of these viewpoints have some merit and individual circumstances should determine which approach is adopted.

It is therefore clear that the security of a well-designed cryptosystem does not depend on the secrecy of the algorithm. The security depends on how that algorithm uses the available secret information, that is, the secret key. The security of any algorithm is usually expressed in terms of the number of bits in the key. This is because an attack open to any eavesdropper is simply to try every possible key. This kind of attack is known as an exhaustive key search

and is an extremely important concept for assessing the security of a cryptosystem. An exhaustive key search of a 40-bit secret key system requires about $2^{40} \approx 10^{12}$ operations to determine the correct key. This is well within the capabilities of modern computer systems and data protected by only 40 bits of secret key should be considered extremely vulnerable. We shall return to this 40-bit figure when we discuss some of the political implications of using cryptography.

2.3 Design principles for secret-key cryptosystems

Before going further it is useful to have some idea how good cryptosystems are designed. Again, for convenience, we shall limit our attention to secret-key systems although some of the comments here also apply to public-key systems.

Some of the most important principles of good cipher design were enunciated by Claude Shannon over 50 years ago. In particular he introduced the concepts of confusion and diffusion [1]. What we would like to arrange is that very little information about either the key or the plaintext is evident in the ciphertext. This is the principle of confusion. In terms of the key, it aims to ensure that virtually all of the key is used to produce even very short ciphertexts so that every encrypted message character will depend on most, ideally all, of the key. The idea behind this is that it should force the eavesdropper to find the whole key rather than being able to find pieces of the key. Diffusion is a little different and aims to produce the situation where every digit of the secret key or the plaintext affects many digits of the ciphertext. Changing just one digit of either the key or the plaintext should produce a very different ciphertext.

Diffusion and confusion are commonly obtained by chaining together smaller ciphers that have good local confusion and diffusion. Each of these little ciphers has its own key. For cascade ciphers these keys are independent but for a product cipher these keys are derived from the same secret key using a key-schedule algorithm. An example of a product cipher is DES and triple DES is an example of a cascade cipher. We shall take a look at DES in the next section and see how the principles of confusion and diffusion are achieved in a cryptosystem that is widely used today.

The aim of these two principles is to ensure that an eavesdropper's fastest method of attack is an exhaustive key search. If this is achieved then the strength of the cryptosystem can be improved by increasing the length of the secret key with an increase in security that is exponential in key length. The security of the system in this context is equivalent to the time it takes for the eavesdropper to discover the key. Adding an extra digit to the secret key effectively doubles the time it takes for the eavesdropper to perform an exhaustive key search.

2.4 The data encryption standard

The data encryption standard (DES as it is commonly called) is one of the most famous secret-key systems in current use. It was designed in the early 1970s and was adopted as a national standard in 1977 by the American National Bureau of Standards (NBS) (now known as the National Institute of Standards (NIST)) [2]. Although numerous attempts have been made to find weaknesses in DES, no serious flaws have been found. Despite this intensive investigation some cryptographers remain uneasy about the true security of DES. This is largely because DES was designed with the involvement of the US National Security Agency (NSA) (the American equivalent of GCHQ), and the design principles for some of the cryptographic elements have never been revealed. However, it is also true to say that almost every modification made to these particular elements of DES results in a weaker cipher. It would appear that the involvement of the NSA has resulted in a strengthening of DES rather than the converse.

The basic structure of DES is shown in Fig 2. It is made up of two kinds of cipher repeated a certain number of times. Other secret-key cryptosystems have the same structure. The first transformation is an involution cipher, an involution being a permutation that is its own inverse, which takes the data X and the key K and produces a ciphertext Y with the following properties:

$$Y = \Gamma(X, K)$$

$$X = \Gamma(Y, K) = \Gamma(\Gamma(X, K), K) \quad \dots (1)$$

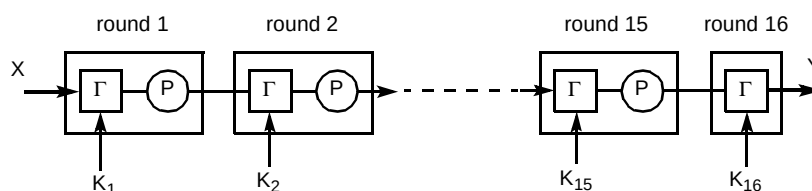


Fig 2 The basic structure of DES. Each round, apart from the last, consists of two ciphers. The encryption and decryption algorithms are the same. Decryption is achieved by reversing the key schedule.

The second transformation is a simple unkeyed permutation which has the following properties:

$$Y = P(X)$$

$$X = P(Y) = P(P(X)) \quad \text{..... (2)}$$

These ciphers are their own inverses so that the encryption and decryption algorithms are the same and to decrypt one simply reverses the key schedule.

As we can see from Fig 2, DES consists of 16 rounds of encryption. There is a very good reason for this as we shall discuss later. To show that DES decrypts correctly we rely on the reversibility properties of the individual ciphers given in equations (1) and (2). If we label the data after each round of encryption as X_j and the data after each round of decryption as Y_j we can write the encryption equations as:

$$X_j = P(\Gamma(X_{j-1}, K_j)) \quad \text{for } 0 < j < 16$$

$$X_{16} = \Gamma(X_{15}, K_{16}) \quad \text{..... (3)}$$

and the decryption equations as:

$$Y_j = P(\Gamma(Y_{j+1}, K_{17-j})) \quad \text{for } 0 < j < 16$$

$$Y_{16} = \Gamma(Y_{15}, K_1) \quad \text{..... (4)}$$

and where X_0 is the initial plaintext and Y_0 is the ciphertext to be decrypted. The output from the decryption process is given by equation (4) and iterating back to the first round of decryption we obtain a set of nested functions of the form:

$$Y_{16} = \Gamma(P(\Gamma(P \dots \Gamma(P(\Gamma(Y_0, K_{16})), K_{15}), \dots, K_2)), K_1) \quad \text{..... (5)}$$

Now we know that the final term from the encryption process, i.e. the ciphertext, is the input to the decryption process so that:

$$Y_0 = X_{16} = \Gamma(X_{15}, K_{16}) \quad \text{..... (6)}$$

The central term in equation (5) can now be written as:

$$Y_1 = P(\Gamma(Y_0, K_{16})) = P(\Gamma(\Gamma(X_{15}, K_{16}), K_{16})) \quad \text{..... (7)}$$

which from the reversibility conditions (1) become:

$$Y_1 = P(X_{15}) \quad \text{..... (8)}$$

but, from equation (3) and the reversibility conditions (2), this can be written in the form:

$$48 \quad Y_1 = P(P(\Gamma(X_{14}, K_{15})) = \Gamma(X_{14}, K_{15}) \quad \text{..... (9)}$$

By similarly iterating through the entire structure and using the reversibility conditions (1) and (2), we eventually arrive at the expression:

$$Y_{16} = \Gamma(\Gamma(X_0, K_1), K_1) = X_0 \quad \text{..... (10)}$$

which is just the plaintext. There is nothing magical about the number 16 as far as the basic structure is concerned, although for DES this number of rounds was chosen for a very specific reason. There is a kind of cryptanalytic attack known as differential cryptanalysis [3] which analyses slight differences in known plaintext-ciphertext pairs and garners information from how they propagate through the cipher. It turns out that 16 rounds is the minimum number of rounds, for DES, to ensure that exhaustive key search is quicker than differential cryptanalysis. In general terms the number of rounds should be such that good global diffusion is obtained and that attacks such as differential cryptanalysis are rendered less effective than exhaustive key search.

The basic structure of Fig 2 is only one of a range of structures that give effective reversibility, that is decryption. It is not difficult to conceive of a complicated series of mathematical operations that would give good diffusion and confusion. The trick in designing a cipher system is to ensure that decryption is also easily achieved. The only requirement for successful reversibility of the entire cipher of Fig 2 is that the structure is symmetric about the mid-point, which in the case of DES is the involutory permutation P in round 8.

Let us examine each round now in more detail. Each round consists of an involution cipher and an involutory permutation. The involution cipher is a common structure in cipher systems and is known as a Feistel cipher. The involutory permutation consists of simply swapping the halves of the data. One round of DES is shown in Fig 3.

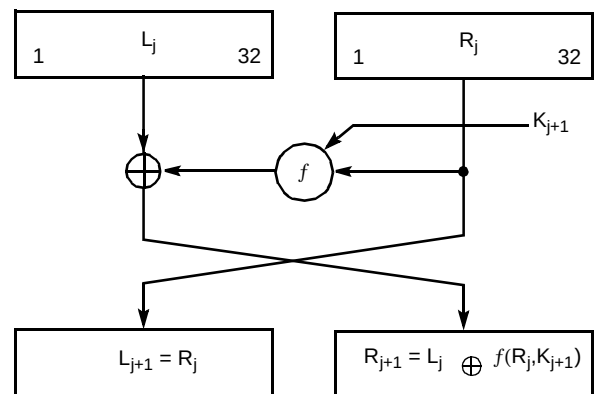


Fig 3 A single round of DES. An input of 64 bits is split into two 32-bit blocks and fed into the Feistel structure. The appropriate key is fed into the nonlinear function f from the key schedule algorithm. After the Feistel cipher the output halves of the data are simply swapped ready for the next round. The operator $\oplus$ is the binary addition operator.

It is obvious that the procedure of swapping halves is an involutory permutation and is its own inverse. It is not so clear, however, that the Feistel cipher guarantees reversibility whatever the details of the function f . To see this we denote the inputs and outputs from the Feistel cipher as (L_j, R_j) and (L'_j, R'_j) , respectively. Feeding this output back into the Feistel structure we obtain (L''_j, R''_j) for the outputs of this operation as:

$$\begin{aligned} L''_j &= L'_j \oplus f(R'_j, K_{j+1}) \\ R''_j &= R'_j \end{aligned} \quad \text{..... (11)}$$

but we have that:

$$\begin{aligned} L'_j &= L_j \oplus f(R_j, K_{j+1}) \\ R'_j &= R_j \end{aligned} \quad \text{..... (12)}$$

so that using these two equations we find that the final output equals the input:

$$\begin{aligned} L''_j &= L_j \oplus f(R_j, K_{j+1}) \oplus f(R_j, K_{j+1}) = L_j \\ R''_j &= R_j \end{aligned} \quad \text{..... (13)}$$

Most of what we have examined in DES so far has, in some sense, almost a peripheral cryptographic significance. It is the nonlinear function f that is the cryptographic heart of DES. A poor choice of this function leads to a very weak system. A good choice of f will lead to very good local confusion and diffusion. Further diffusion occurs throughout the rounds because of the swapping of halves and the binary addition operator $\oplus$ in the Feistel cipher.

The nonlinear function f takes as its inputs 32 bits from the right half of the 64-bit input data block and 56 bits of secret key from the key-schedule algorithm. The last byte of the key is deleted and the 32-bit data block undergoes a linear expansion to become a block of 48 bits. The 48-bit key and this data block are added together modulo 2 and the result of this is fed into the eight substitution, or S-boxes of DES. Each S-box takes as its input 6 bits and gives 4 bits as its output. This output is then subject to a further linear permutation. The resultant 32-bit data block, the output from the function, is the half word to be added to the left half word in the Feistel cipher of Fig 3. The basic structure of this function is in depicted Fig 4 and the operation of an

S-box is shown in Fig 5. The S-boxes contain all the nonlinearity of DES. Their design is critical to the overall strength of the algorithm. Even slight changes to the permutations they perform can seriously weaken DES.

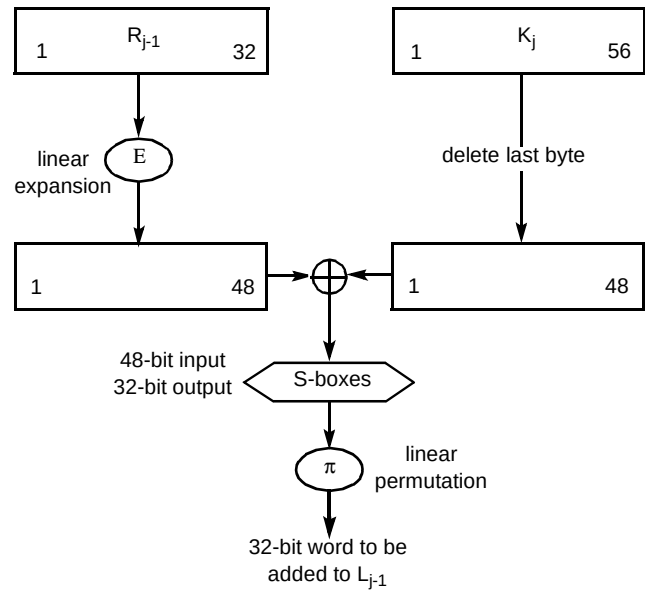


Fig 4 The nonlinear function of DES. The eight S-boxes contain the non-linear transformations and are the cryptographic heart of DES. Each S-box can itself be considered to be a block cipher.

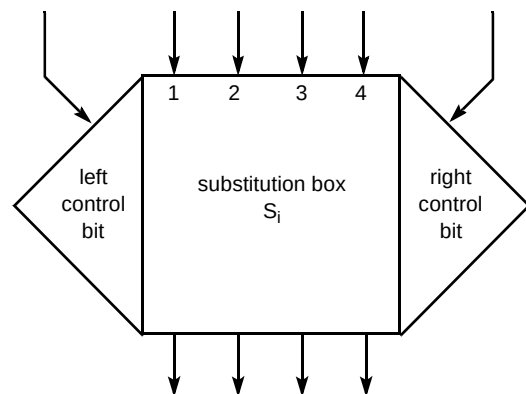


Fig 5 The basic structure of the input and outputs to an S-box.

Table 1 The look-up table for the first S-box of DES.

Control bits		Half-byte value of the input bits															
Left	Right	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
1	0	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
1	1	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Bits number 1 and 6 of the input to an S-box are control bits and a different permutation is performed depending on which of the four possible combinations of control bits is received. The look-up table for the first S-box of DES is shown in Table 1. We can see that if the input bits are 011001 the control bits are 01 and the half-byte value of the remainder, that is $1100 \equiv 12$ so that the output value is simply $9 \equiv 1001$.

The full details of all the other S-boxes, the linear expansion function and permutation can be found in any good textbook of cryptography [1, 4–6] and are not relevant to our discussion here. The S-boxes are the least understood component of DES and there are perennial fears that they contain a trapdoor. Trapdoors are often inserted into algorithms to weaken them sufficiently for export. The fears over whether DES has a trapdoor stem from concern over the involvement of the NSA in the design of DES and the fact that the design principles for the S-boxes have never been revealed. However, DES is probably the most extensively studied cryptosystem and despite this intensive effort over the last 20 years to find weaknesses or trapdoors none has ever been found.

We now have an almost complete picture of DES. The basic structure consists of the repeated application of smaller ciphers. The fundamental components of DES are the S-boxes which provide local confusion and diffusion. This local effect is ‘amplified’ by repeated application of the Feistel cipher and the swapping of halves permutation. There is one critical component of DES about which we have said very little. This is the key. DES keys are 64 bits long although 8 bits of this are discarded so that the effective key length for DES is just 56 bits. The key is fed into a key-schedule algorithm. This algorithm provides the 16 sub-keys required for the f function in the Feistel cipher in each round. It would be inappropriate to go into detail but this key expansion algorithm produces a different 56-bit key for each round. The exact details of the algorithm can be found in good cryptography texts [1, 4–6] or the standard itself [2]. It is important to note that the key expansion algorithm does not increase the security of DES in the sense that an exhaustive key search still requires a search of a key space of 2^{56} bits.

DES is an enormously important cryptosystem and is still in widespread use. Many ATM transactions are protected by DES. However, for large sums of money DES is no longer thought to provide adequate protection. This is not because of any supposed weakness in DES itself but that the key is no longer thought to be large enough to withstand a concerted attack. It would require a great deal of resources to be able to crack a 56-bit DES system but the benefit of so doing may run into millions. APACS, the Association for Payment and Clearing Services, for example, handles something like £26,000 billion worth of electronic transactions a year. Where these sums of money are

concerned it is not surprising that 56-bit cryptosystems are not considered adequate and are, consequently, not in use. In the next section we take a brief look at a cascade cipher based on DES. It is a way of enlarging the key size of DES. This cipher is known as triple-DES and is also used extensively.

2.5 Triple DES

When DES was designed back in the early 1970s 56 bits of key seemed perfectly secure. After all, performing 2^{56} operations in a reasonable time is not a trivial exercise even today. However, while it is not a trivial exercise, it is certainly not unfeasible and dedicated processors can be manufactured to improve the speed of an exhaustive key search operation. Consequently for applications where a high degree of security is required, such as the electronic transfer of large sums of money, DES with a 56-bit key is no longer considered safe.

One obvious way to increase the key space is simply to chain two DES systems with different keys together. Intuitively we would expect this to be no weaker than single-key DES but now with a 112-bit key. Unfortunately as so often happens in cryptography our intuition fails and this turns out not to be the case. The problem here is not so much with DES itself but with the notion of multiple encryption.

Multiple encryption is susceptible to an attack known as the ‘meet-in-the-middle’ attack first invented by Merkle and Hellman [7]. To see how this kind of attack works we suppose that we shall encrypt the plaintext X to produce the ciphertext $W = E_1(X, K_1)$. This is then further encrypted under a different cipher to produce the ciphertext $Y = E_2(W, K_2)$ where K_1 and K_2 are independent sub-keys of bit length N_1 and N_2 , respectively. We shall also suppose that the attacker has one valid plaintext-ciphertext pair (X_0, Y_0) and that she also knows a few other valid ciphertexts. The attack is as follows.

The first step is to decrypt Y_0 for each of the possible values of the key K_2 . This requires 2^{N_2} decryptions. This gives 2^{N_2} pairs of the form (W, K_2) . The next step goes in the other direction and for each of the 2^{N_1} possible values of K_1 encrypts X to find its corresponding W . This is then checked to see if there are entries in the list from the first step. If a match is found this is further checked by decrypting the valid ciphertexts with the key (K_1, K_2) to see if they yield valid ciphertexts. If they do then the attack has uncovered the key.

The first step uses at most 2^{N_2} decryptions and step two requires at most 2^{N_1} encryptions. Thus the effective key size of the cascade cipher is approximately the larger of the two keys (K_1, K_2) . Simply repeating a cipher with key size K bits n times increases the effective key size to about

$nK/2$ rather than nK . Thus to increase the effective key size n should be chosen to be 3. However, if the cipher is a group even this increase is non-existent since because of the group properties any sequence of group operations would be equivalent to a single operation. When designing ciphers considerable effort goes into ensuring that the cipher is not a group. It was shown in 1992 that DES is not a group [8].

In practice the following scheme shown in Fig 6 is used to increase the effective key space of DES [9]. This cryptosystem is known as triple-DES and has two independent sub-keys so that its effective key size is thought to be 112 bits. The data is first encrypted under key K_1 , then decrypted under key K_2 and finally encrypted under key K_1 . The final output is therefore given by $Y = E(D(E(X, K_1), K_2), K_1)$ where E and D represent encryption and decryption, respectively. It is not, however, a cascade cipher in the sense that not all sub-keys are independent. It should also be noted that triple DES should be thought of as a cipher system in its own right and not just as DES repeated 3 times. There have been rumours that when analysed in this fashion the actual key space of triple DES is lower than 112 bits, but these rumours may, of course, be just malicious gossip.

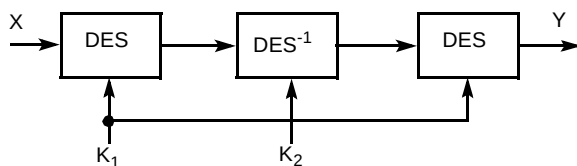


Fig 6 The scheme for encryption using triple DES. This cipher is thought to increase the size of the key used to 112 bits while offering the same strength of cipher as DES.

2.6 A note on stream ciphers

DES, as we have described it, takes an input data block of 64 bits and uses a key of 56 bits to encrypt that data to produce 64 bits of ciphertext. This is an example of a block cipher. Suppose there were repeated plaintext patterns that were to be encrypted under a block cipher with the same key. Such patterns can occur with standard letter formats, for example. These patterns would be repeated in the ciphertext under a block cipher and can be exploited. This mode of operation of a block cipher is known as electronic code book (ECB) mode. It is not recommended that any block cipher be operated in this mode [1, 4-6].

To overcome the problem of patterns in the ciphertext we need to ensure that, in effect, each new block is encrypted under a different key so that patterns in the plaintext will not be reflected in the ciphertext. A cipher which is encrypted bit by bit, or symbol by symbol, is called a stream cipher. By operating DES in a different mode it can be made to behave as if it were a stream cipher thus

eliminating the problem of patterns in the ciphertext. The one time pad is a stream cipher and takes a running key and performs a bit-wise exclusive-or of this with the message. The one time pad is theoretically unbreakable. Stream ciphers aim to emulate this by using a shorter secret key to generate a longer stream of pseudo-random data which forms the running keystream. In Fig 7 we illustrate the operation of a 64-bit block cipher in output feedback (OFB) mode. This technique converts the block cipher into a stream cipher.

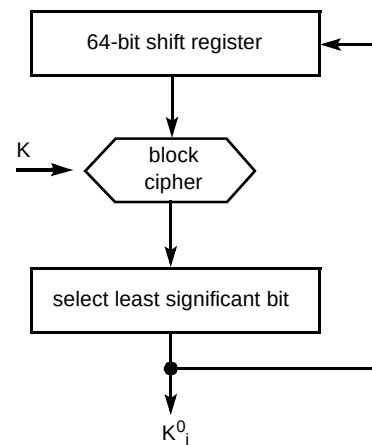


Fig 7 The operation of a block cipher in output feedback (OFB) mode. The register is shifted 1 bit left after each encryption so that the output bit forms the rightmost bit of the register. The output bits are now used as a running keystream.

Running a block cipher in OFB mode converts it to a stream cipher with no error propagation and with more encryptions per bit. The initialisation vector of the shift register does not need to be kept secret, but it is preferable that it is not revealed. By selecting the first n bits from each encryption in OFB mode this technique can generate a running keystream consisting of n -bit symbols. Encryption and decryption in this mode are, of course, the same.

There are other techniques for converting block ciphers into stream ciphers as there are many other ways of designing stream ciphers. The subject of stream cipher design is a specialist field in its own right [10] and it is not appropriate here for us to go into a discussion of the relative merits of block and stream ciphers. However, it is important to understand that even very strong ciphers, such as DES, may not be as secure as we might think when operated in ECB mode.

2.7 A final word

Despite its advancing years there is still life in DES and it has successfully fended off a multitude of attacks. DES, of course, is only a very small part of the story of secret-key algorithm design. It does, however, highlight several important features. In particular we have seen how, using sophisticated permutations and substitutions DES achieves

the goal of confusion and diffusion. We have also seen how DES is built up from smaller reversible components each of which is a cipher. Many of these kinds of features are found in other secret-key algorithms. The secrecy is obtained by a complex series of mathematical shufflings which are very hard to undo without the key.

The issue of key size is a particularly important one. A very strong algorithm, such as DES, can be made very much weaker by restricting its key space in some way. Similarly the overall security of DES can be improved by multiple encryption which has the effect of increasing the key size. Key size is an important commercial issue because most governments will only allow the export of secret-key systems which have an equivalent strength of a 40-bit system. Where the majority of the world's software is exported from the US, this has implications for the use of US products for security. We shall return to the thorny problem of export controls in the final section.

The issue of key size and security becomes further complicated when we consider public-key systems in the next section. Public-key systems derive their security from quite different fundamental precepts to secret-key systems. One of the consequences of this is that key sizes for equivalent security to secret-key systems are vastly different. This is further complicated by the fact that public-key and secret-key systems are often used together as in the encryption tool PGP (pretty good privacy).

Secrecy and authenticity are independent aspects of a cryptosystem. For convenience we have focused on secrecy. Secret-key systems can also be used to provide authentication services but a discussion of these techniques falls outside the scope of this review. We shall return to the issue of authentication when we have taken a brief tour of public-key cryptography. It is in the area of authentication that public-key cryptography is truly revolutionary and gives us the most important techniques for ensuring the integrity of electronic transactions.

3. Public-key cryptography

3.1 Secret keys versus public keys

Secret-key and public-key cryptosystems are very different beasts. Secret-key systems rely upon a complex series of permutations and substitutions to achieve security. The security of public-key systems is entirely dependent on the unproven difficulty of reversing certain simple mathematical operations. The trust in secret-key systems is one of trust in the key distribution mechanisms; each user must possess a copy of the secret key. In public-key cryptography everyone can possess an individual's public key and must do so to encrypt a message. The trust in public-key systems is trust in the authenticity of the key; each public key should be uniquely identified with a particular user.

Public-key cryptography arose in the 1970s [11, 12] in response to the problem of key distribution in secret-key systems. Instead of having just one key each user generates a key pair. The keys are mathematically related in a very particular way. Let us suppose that the public key K_{pub} can be generated simply from the private key K_{pri} by some mathematical function f so that:

$$K_{pub} = f(K_{pri}) \quad \text{..... (14)}$$

The basic idea behind public-key cryptography is that the inverse problem, calculating K_{pri} from K_{pub} should be prohibitively difficult. An example of a simple mathematical operation with this property is multiplication and its inverse. Multiplication of two large prime numbers p and q to give m can be performed very quickly. However, if we are simply given m and asked to find p and q we are faced with a problem of an entirely different magnitude. Even the very best factoring algorithms can take many years for large m whereas the multiplication to produce m may have taken a few seconds. The public key can therefore be published widely without fear that the private key can easily be recovered from it.

If Alice wishes to communicate in secret with Bob using a public-key cryptosystem she obtains Bob's public key and encrypts the message with it. The cryptosystem should be such that only someone with the private key, that is, Bob, can decrypt the message. The main flaw with public-key cryptography is that the public key must be trusted. Alice must be certain that she is using Bob's public-key and not the key of someone masquerading as Bob. The problems associated with secret-key distribution have just been shifted elsewhere.

The real significance of public-key cryptography for electronic commerce is not in its ability to provide secrecy but in its ability to provide authenticity. If Bob wishes to prove to Alice that a particular document is from him and has not been altered he simply takes a copy of the document and encrypts it with his private key. He then sends both the encrypted and the original document to Alice. Alice uses Bob's public key to decrypt the document and checks to see if both copies match. If they do, she accepts it as a genuine unaltered message from Bob. The encrypted document is known as a digital signature and is the electronic equivalent of its handwritten counterpart. Unlike the handwritten version, however, a digital signature has a number of important properties which make it much stronger. We shall examine digital signatures in a little while. For now we shall see how two popular public-key systems work. A full understanding of these systems requires some number theory which is beyond the scope of this paper. However, some details are essential if we are to understand the fundamental difference between public-key systems and secret-key systems.

3.2 Diffie-Hellman key exchange

The first public key system to be invented was the Diffie-Hellman key distribution system [11]. This exploits the properties of a conjectured one way function (OWF) which is modular exponentiation over a finite field [13]. A one way function, f , is a function $f: A \rightarrow B$ such that for all $x \in A$, $f(x)$, is easy to compute, but for almost all $y \in f(A)$ it is prohibitively difficult to find the corresponding x such that $y = f(x)$. Diffie and Hellman conjectured that the following function is one way — a conjecture, incidentally, which has not been proven in a strict sense.

The input x to the function is taken from the finite field $\mathbb{Z}_{p-1}$, that is from the integers $\{0, 1, 2, \dots, p-2\}$. The input is mapped by the function to the group of units of $\mathbb{Z}_p$ which is labelled $\mathbb{Z}_p^*$. The number p is a large prime such that $p-1$ has a large prime factor so that $\mathbb{Z}_p^*$ is simply the set of non-zero elements of the Galois field $\text{GF}(p)$. The OWF is given by:

$$f(x) = \beta^x \quad \text{in } \text{GF}(p) \quad \text{..... (15)}$$

where β is a primitive element of $\text{GF}(p)$. A primitive element is an element of multiplicative order $p-1$. The order of an element of a group is the smallest number of times it must be multiplied to itself to equal the identity element. If no such number exists the order of that element is taken to be infinite. Basically a primitive element is a generator of the group and the function (15) cycles through each of the possible values. As an example, consider the element 3 of $\text{GF}(7)$. We have the following in $\text{GF}(7)$:

$$3^1 = 3, 3^2 = 2, 3^3 = 6, 3^4 = 4, 3^5 = 5, 3^6 = 1 \quad \text{..... (16)}$$

By restricting the operations to those on primitive elements we ensure that the output samples the entire space and does not form a sub-group of lower order. The number of primitive elements of the finite field $\text{GF}(p)$ is given by $\phi(p-1)$ where ϕ is the Euler totient function. For prime p Euler's function is simply given by $\phi(p) = p-1$ which gives us some idea of how many primitive elements there are for a given Galois Field. Alice and Bob can use this function on this algebraic system to exchange a secret without ever having met. This is how it is done.

Alice and Bob agree, not necessarily in secret, on a common modulus p and a common primitive element b . They each independently choose a random, secret number known only to themselves. We shall call these numbers x_A and x_B , respectively. Using these numbers they compute, in $\text{GF}(p)$, the information:

$$y_A = \beta^{x_A}; y_B = \beta^{x_B} \quad \text{..... (17)}$$

and publish the numbers y_A and y_B in a public directory. Alice now obtains y_B from this directory and Bob likewise obtains y_A . Alice now computes, in $\text{GF}(p)$:

$$K_A = y_B^{x_A} = \beta^{x_B x_A} \quad \text{..... (18)}$$

and Bob computes, in $\text{GF}(p)$:

$$K_B = y_A^{x_B} = \beta^{x_A x_B} = K_A \quad \text{..... (19)}$$

Alice and Bob now share a common number which can be used as a key in a secret-key cryptosystem. If Eve, the eavesdropper, wishes to find this common number, she must attempt to reconstruct it from knowledge of y_A and y_B . Assuming she knows the common modulus and primitive element this entails her being able to invert the OWF. Inversion of the Diffie-Hellman OWF is the well-known discrete logarithm problem and for large numbers of the order of $p \approx 10^{100}$ appears to be an extremely difficult problem requiring years on a computer capable of performing 104 mips. It also appears that breaking the Diffie-Hellman system is equivalent to solving the discrete logarithm problem. There are, however, no formal proofs of these two statements.

As we have already mentioned the weak link in such a system is the ability of an attacker to masquerade as either Bob or Alice. An authenticated Diffie-Hellman system where each user can be authenticated to every other user is therefore essential for the secure operation of this kind of key exchange. An example of this kind of authenticated exchange is the station-to-station protocol [14].

The security of the Diffie-Hellman key exchange is based upon the supposed and unproven difficulty of finding discrete logarithms. Modular exponentiation is by contrast a very quick process and there exist very fast algorithms to achieve this. The security achieved in public key systems is based upon very different principles to those of secret key systems which rely on complex mathematical permutations to achieve confusion and diffusion. Cryptographers using public-key cryptography have to make the assumption, albeit a reasonable one, that advances in mathematics will not spectacularly change the ability of an attacker to compute these difficult problems. The reasoning is based on complexity classes and the theoretical computing time that is required to solve a particular problem. It seems quite likely that the discrete log problem and the problem of

factoring belong to a class of problems that are, in a computational sense, 'hard' problems.

3.3 The RSA cryptosystem

In 1978 Rivest, Shamir and Adleman published a public-key cryptosystem which has become known as RSA [12]. This cryptosystem relies on the supposed difficulty of factoring and, like the discrete log problem used in Diffie-Hellman key exchange, no formal proof of its difficulty exists. Each user calculates a key pair consisting of a private and public key. The public key is placed in some trusted public directory and used to encrypt messages for that user.

The RSA encryption and decryption functions [12, 13] are given by:

$$\begin{aligned} f(x) &= x^e \\ f^{-1}(y) &= y^d \end{aligned} \quad \text{..... (20)}$$

where both power functions are computed in $\mathbf{Z}_m$. The numbers e and d are multiplicative inverses of each other in the finite field $\mathbf{Z}_{\phi(m)}$. The number m is the product of two large distinct primes, p and q . Euler's function for $m = pq$ is given simply by $\phi(m) = (p-1)(q-1)$. To encrypt, therefore, the data, x , is raised to the power of e modulo m to form the ciphertext y . Decryption takes y and raises it to the power of d modulo m .

If Alice wishes to send an encrypted message to Bob she must first obtain m and e from Bob, or from some trusted directory. The pair of numbers (m, e) are Bob's public key. As we have seen, to decrypt Bob must take Alice's ciphertext and raise it to the power of d modulo m . The two numbers (m, d) form Bob's private key.

In order to see where the security comes from we recall that the pair (e, d) are multiplicative inverses of each other in the field $\mathbf{Z}_{\phi(m)}$. In order to generate a private key Bob takes his prime numbers p and q and calculates the value of Euler's function $\phi(m)$. He then randomly selects an e such that $1 \leq e < \phi(m)$, and checks to see whether this is a unit of $\mathbf{Z}_{\phi(m)}$, i.e. he checks to see whether this element has an inverse. If so, this is chosen as e and (m, e) becomes the public key. He then calculates the corresponding inverse d and this, together with m becomes his private key. The algorithms to check for an inverse element of a finite field and to calculate that inverse are very quick. In order to be able to decrypt, an attacker needs to have access to d . This is easy to calculate if e and the value of Euler's function are known. However, in order for an attacker to calculate Euler's function she must know the individual primes p and q . In other words she must be able to divide m into its prime factors. It can be shown that calculating Euler's function $\phi(m)$ for $m = pq$ is computationally equivalent to factoring m .

In order to see that RSA correctly decrypts we must show that:

$$(x^e)^d = x \quad \text{in } \mathbf{Z}_m \quad \text{..... (21)}$$

We shall show this only for the case where x is a unit of $\mathbf{Z}_m$. It does not happen with significant probability that x is not a unit of $\mathbf{Z}_m$. This is because a necessary and sufficient condition for x to be a unit of $\mathbf{Z}_m$ is that $\gcd(m, x) = 1$. However, it can be shown that, even when x is not a unit, RSA correctly decrypts [13].

Suppose then that x is a unit of $\mathbf{Z}_m$. We know that e and d are inverses of each other in $\mathbf{Z}_m$ so that we can write:

$$\begin{aligned} e \cdot d &= 1 \pmod{\phi(m)} \\ \Rightarrow e \cdot d &= Q\phi(m) + 1 \pmod{m} \end{aligned} \quad \text{..... (22)}$$

The decryption function can now be written as:

$$\begin{aligned} x^{e \cdot d} &= x^{Q\phi(m) + 1} \\ &= x \cdot x^{Q\phi(m)} \\ &= x \cdot (x^{\phi(m)})^Q \end{aligned} \quad \text{..... (23)}$$

where all calculations are performed mod m . We now make use of Euler's theorem which states that if u is a unit of $\mathbf{Z}_m$ then:

$$u^{\phi(m)} = 1 \pmod{m} \quad \text{..... (24)}$$

Using this in equation (23) we find that equation (21) is valid and that RSA correctly decrypts when x is a unit of $\mathbf{Z}_m$.

There are a number of things to note about RSA. The key size, effectively the size of the product, needs to be much larger than those of secret-key systems for equivalent security. The current recommendations are that at least 512-bit keys are used. This is based upon the estimated difficulty of factoring a 512-bit number into its composite primes. All of the sieving algorithms for factoring proposed so far have running times, proven or conjectured, which vary with the input m as:

$$A \exp(\lambda \sqrt{\ln m \ln(\ln m)}) \quad \text{..... (25)}$$

where A and λ are constants. Roughly speaking increasing the size of m by 15 decimal digits increases the factoring time by a factor of ten.

It is important to note that while being able to factor large numbers would certainly give the ability to break RSA, it has not been proven that factoring is the fastest method of attack. In other words it is not known whether breaking RSA is equivalent to factoring. Indeed, as we have

stressed, it is not proven that factoring is a 'hard' problem. We do not, therefore, have any mathematical guarantees that fast factoring algorithms will not be developed or that another way of breaking RSA will not be found. However, both the general area of factoring and RSA itself have been the subjects of intense study and our conjectures about the difficulty of breaking RSA and factoring seem to be borne out at least by the empirical evidence to date.

This difficulty of factoring also implies something about the ease with which prime numbers can be found. The primes Bob generates must be kept secret so that, if a deterministic method for prime generation is used, knowledge of this might allow an attacker to quickly generate Bob's primes for herself. There are basically two methods open to Bob to generate primes. He can generate a randomly chosen probable prime or a probably random prime. The first method involves choosing a number at random and using tests for primality which give probabilistic answers. By repeating these tests sufficiently many times he can be reasonably confident that his number is prime. The other method of prime generation involves generating a number that is definitely prime but is probably randomly chosen.

3.4 The use of public-key cryptography

The public-key techniques we have described are beautiful applications of number theoretic concepts and make very elegant cipher systems. However, elegance, while satisfying, is hardly our primary concern in designing or using a cipher system. One of the main obstacles to the use of RSA for encryption, for example, is its speed. In general secret-key algorithms perform an order of magnitude quicker than public-key algorithms. For this reason secret-key systems and public-key systems are often used together in hybrid systems. The most celebrated example of this is PGP. This was written by Phil Zimmerman and he combined RSA and the IDEA algorithm to create a strong cryptosystem [15]. Basically the public-key component of the algorithm is used to exchange a 128-bit secret key for use in IDEA. The data is then encrypted using IDEA which is thought to be an extremely strong cipher [4]. PGP has become a common tool for encryption of personal e-mail and is also used extensively in mixmaster anonymous remailers. It is not an appropriate tool for use within large organisations in its current form, however, because it relies on a questionable system of key management.

With slightly different key management systems PGP would be a very strong cipher appropriate for use within corporate organisations. We shall address the question of key-management and public-key systems in the next section.

4. Digital signatures and trusted third parties

4.1 Digital signatures

The practice of cryptography either for confidentiality or for authenticity is about shared secrets. Two initially disjoint universes must be correlated in some way before secure communication can take place. In secret key cryptography these universes are correlated by key distribution so that Alice and Bob possess the same secret. Public-key cryptography was designed to overcome the problem of key distribution. However, it cannot of itself provide the correlation required for these universes to share a secret. In public-key systems this problem manifests itself in the authenticity of the key. If Alice is to use Bob's public key safely she must have confidence that it is indeed the public key (or one of the public-keys) belonging to Bob. This entails both an identity check and an integrity check. We want to be certain both that the key is Bob's and that it has not been altered. Surprisingly, public-key cryptography helps us to find a solution to these problems in the form of digital signature techniques.

A digital signature is the electronic equivalent of its handwritten counterpart. In a conventional signature we would attach a private 'secret', our signature, to a document. More specifically we implicitly make the assumption when writing and accepting signatures that a signature is unique to the signer and no-one else knows how to reproduce it. Digital signatures achieve a similar goal. A digital signature is simply a piece of data attached to an electronic document. We shall examine exactly how a digital signature is constructed in a moment. In general, digital signatures are far stronger than handwritten signatures and possess additional useful properties. Given the obvious inadequacies of handwritten signatures it is surprising that digital signatures have not yet enjoyed a more widespread use.

There are several digital signature schemes based on public-key cryptography available for use. We shall look at just one scheme based on RSA. In order to sign some data using RSA Bob would encrypt a copy using his private key forming the signature, which we called ciphertext previously, as:

$$S(x, d) = x^d \mod m \quad \text{..... (26)}$$

He would then attach x to S and send the resultant data to Alice. To check the signature Alice would obtain Bob's public key and raise the signature to the power of e modulo m so that:

$$S^e = (x^d)^e = x^{e \cdot d} = x \mod m \quad \text{..... (27)}$$

If the two copies of the data Alice possesses are identical she accepts this as a valid signature.

There are several things to note about this signature:

- it is almost unforgeable — to forge this kind of signature an attacker would have to be able to break RSA,
- it is bound to both the identity of the signer and the data itself — if Bob were to sign another piece of data the signature would be different but still provably Bob's,
- Bob cannot later deny his signature without claiming that his key has been compromised — this non-repudiation is an important feature of a digital signature,
- it cannot be copied and appended to another document,
- anyone with access to Bob's public key can verify the signature,
- it provides an integrity check on the data — if the data is altered in any way it will be evident when an attempt is made to verify the signature,
- an independent third party can verify the signature.

These features make digital signatures a very attractive technology option for authenticity.

In practice, digital signatures are not made by encrypting entire documents. The speed limitations on public-key cryptosystems would make the process intolerably slow. In order to overcome this problem a digest, or hash of the message is created. A hash function takes a variable amount of input data and compresses it down to a fixed length. For example, the secure hash algorithm (SHA) [16] for use in the US digital signature standard (DSS) [17] takes data and hashes it down to 160 bits. The hash function is so designed that even changing one bit of the input creates a very different hash value. Of course there is a small probability that two documents will hash to the same value. However, this is a very small probability for a well-designed hash function. It is this hash value that is encrypted with the private key and appended to the data to form the signature.

To verify the signature Alice decrypts the signature and checks the result with the hash value of the document. Calculating hash values is a very quick process. The procedure for creating a digital signature is shown schematically in Fig 8.

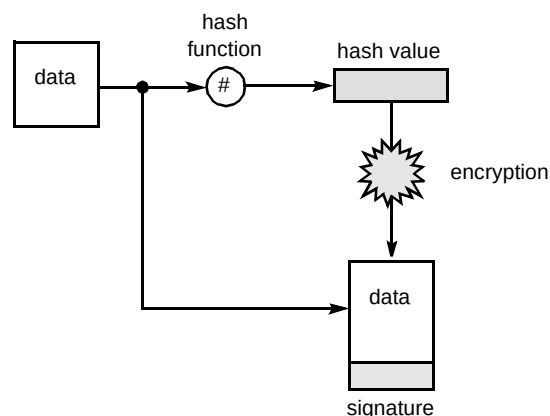


Fig 8 To create a digital signature a digest is made of the data. This digest, or hash, is encrypted with the private key of the signer and then appended to the original data. If either the data or the signature is altered they will no longer agree when an attempt to verify the signature is made.

Any electronic data can be signed. The beauty of using a digital signature is that once the data has been signed with a trusted key it carries its own certification. In other words such data can be passed and stored on open networks. The trust to a large extent has been transferred to the data rather than the network infrastructure. Trusted data can be stored anywhere. One particular example is the storage of personal certificates and other information on smart cards. The potential applications are many. Another example is the signing of software so that only properly signed software will work in a software suite making the propagation of viruses, for example, much more difficult. It is the ability of digital signatures to allow the trust to be carried by the data that enables us to consider such applications.

Digital signatures are one of the fundamental enabling technologies for electronic commerce. However, we have glossed over the real source of trust in the system. The public keys for verification of signatures must be trusted. In our terms this means that public keys must themselves be digitally signed by an appropriate authority. A signed public key together with the public key of the signing authority is usually termed a certificate. Certificates and the authorities who issue them are dealt with in much more detail by Skevington and Hart [18]. The authorities issuing certificates for public keys are known as certification authorities or (CAs). Generalisation of the concept of a CA leads us to the notion of trusted third parties or TTPs which are the subject of the next section.

4.2 Trusted third parties

Having accepted that public keys have to be trusted and that one way of establishing that trust is to have an independent body sign those keys we are now in a position to generalise the notion. Public keys are simply a specific form of data. Any data can be certified by an appropriate

authority. CAs for certification of public keys are just one example of a class of independent service providers known as trusted third parties or TTPs. A TTP is an entity that is trusted to perform in a certain way given a specific set of instances. There is nothing new in the concept of a TTP and we use many kinds almost daily. Examples of TTPs are banks, lawyers, vehicle licensing centres, etc. However, we tend to restrict the term TTP to those providing trust services over an electronic communications network.

If electronic commerce is to become a reality it must be underpinned by trust. Merchants and customers need to have some form of trust in the transactions they are conducting. That trust can be supplied by a network of TTPs. So, for example, my bank could issue me with an electronic certificate containing my signed public key for use in commercial transactions. Merchants could then verify my transactions firstly by checking my signature and then verifying the bank's certificate of my public key. The public key of the bank could itself be certified by a higher authority in the trust chain and any process of verification would need to trace a path of trust through this chain to a single point of trust which might well be an institution like the EC or the UN. This hierarchy of trust and its possible complexity is illustrated in Fig 9. Note the possible existence of cross certification. In practice, it would not be expected that this hierarchy of trust will be more than two or three layers deep and for most purposes a single certificate should be sufficient.

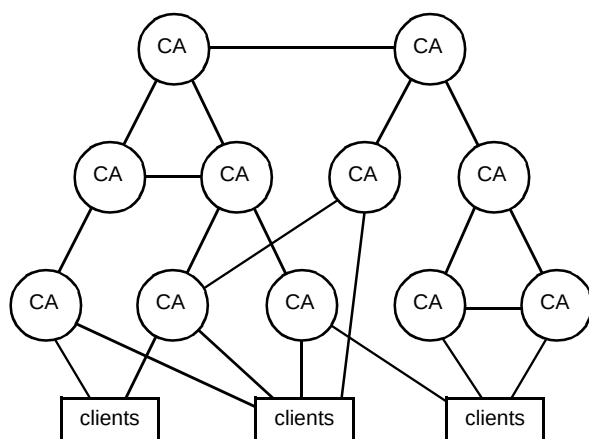


Fig 9 Illustration of the kind of hierarchy of trust that might exist in a future trust infrastructure. Note the possible existence of cross certification. Each chain of verification must be traceable to a single point of trust.

Digital certificates can be stored on smartcards. With the likelihood that future PCs are equipped with smartcard readers every transaction could, in principle, be protected by strong cryptographic mechanisms. We might imagine, for example, the situation where the necessary certificates for driving a vehicle were stored on a smart card. If cars were to be fitted with smartcard readers we could prevent

unauthorised users from driving. Other interesting digital signature and smartcard applications are easy to imagine.

There are a multitude of applications for TTPs and digital signature technology. In this brief introduction we have glossed over a few. TTPs will be a fundamental component of a future global electronic market-place. There are no other practical alternatives for distributing trust. It is probable that large institutions who already function to some extent as trusted intermediaries in our everyday lives may wish to become TTPs. Such institutions might include banks, telecommunications companies, insurance brokers, etc.

The UK government has recognised the importance of having an appropriate trust infrastructure in place to support electronic commerce and is preparing legislation for the licensing of TTPs in the UK. An appropriate licensing regime is likely to reinforce the overall trust in the system and together with an appropriate legal setting for digital signatures should form the basis of a legislative framework for ensuring the success of electronic trading in the UK.

5. Governmental control of cryptography

5.1 Export controls

Cryptography is viewed by most Western governments as a munition and as such subject to stringent export regulations covered by international agreements such as the Wassenaar Arrangement. In order to export products containing cryptographic mechanisms an export license must be sought. In general, the export of cryptosystems for confidentiality is much more difficult than the export of cryptosystems exclusively designed for authenticity. One peculiar effect of these export controls has been that software containing the source code for strong cryptosystems is not exportable whereas a book containing the same source code printed on paper can quite happily move across national boundaries.

In general, the export limits are expressed in terms of key sizes. For secret-key systems only 40-bit systems are allowed for general export. If the assumption is made that export controls exist for national security reasons it follows that 40-bit cryptosystems are trivially breakable by national security agencies. This causes some concern to companies whose operations span national boundaries especially where they operate in jurisdictions whose national security agencies have stated policies of using their skills to favour their own national industries. Actually, as recent work has shown¹, 40-bit cryptosystems should not be considered secure even against adversaries with modest computational resources.

¹ At the beginning of February 1997, a student at Berkeley, Jan Goldberg, managed to crack a 40-bit cipher in just 3½ hours.

It is difficult to compare key sizes for secret- and public-key cryptosystems directly. A comparison can be made by comparing the fastest known algorithms for factoring or finding discrete logs with the time required to perform exhaustive key searches for secret-key systems. Table 2, giving direct key length comparisons between public- and secret-key systems, has been taken from Schneier's book [4].

Table 2 Comparison of key lengths for secret- and public-key cryptosystems for equivalent resistance to brute force attacks.

Secret-key systems (key length in bits)	Public-key systems (key length in bits)
56	384
64	512
80	768
112	1792
128	2304

As a general rule of thumb if a confidentiality system can be exported its strength will be about equivalent to a 40-bit secret-key system or less. However, as 40-bit systems should not be used to protect data of any real value, a home-grown solution to confidentiality is required for protection of valuable data.

Despite most countries employing severe export controls on cryptography, few countries apply import controls or usage restrictions within their own borders. This has led to the situation where US software developers are creating software for use within the US with strong cryptography but crippling that capability when the same product is shipped for export. The export-strength versions offer inadequate protection for transactions of any real value and the security of these products should only be relied upon for the lowest value transactions. Thus I would have no problem in trusting my credit card details, for example, to systems using 40-bit cryptography whereas I would be very wary of using the same cryptosystem for transferring larger sums.

In the UK there are no restrictions on the use of cryptography within the national boundaries but because of the domination of the US in the software market the commercial use of strong cryptography is hampered by the lack of any suitable US products. US companies are continually applying pressure on the US government to relax the export rules because they feel that the severe export regime is damaging their profitability. So far they have met with little success.

Products for authentication are treated very differently. Most governments recognise the importance of a strong authentication framework for electronic commerce. Systems for strong digital signatures, for example, can in general be exported provided they cannot be used to offer confidentiality. Indeed the US government has made its digital signature standard freely available for use in the hope that this will promote its widespread adoption. This standard developed by the NSA seems to be a strong digital signature method that cannot effectively be used for confidentiality.

An interesting question prompted by the 40-bit export limit is how large we should make our key to be safe against attack. It is clear that 40 bits is inadequate. Is 56-bit DES sufficient, for example? Again the answer to this has to be a qualified no because it seems to be perfectly possible to construct computational machines that will uncover a DES key in a matter of hours. The cost of these machines is within the budget capabilities of small-to-medium-sized enterprises. If we take as our benchmark figure the sum of \$300,000,000, that is, the estimated cost of one operation by the US Navy to recover a Russian submarine, we can assume that the NSA, for example, has similar resources available to it for codebreaking. With this kind of financial resource the NSA could well have the capability of finding keys substantially larger than 56 bits. This is, of course, complete speculation, but further details and some hard facts and figures are available to the interested reader [19].

We should not be overly concerned about the doings of national security agencies. They will continue doing whatever they need to to stay ahead of the game. What is of more concern is the increasing ability of organisations with more modest resources to crack cryptosystems. Also of concern are the growing number of recreational codebreakers who attempt to uncover keys for no obvious financial motive. Bad publicity from this kind of unwelcome attention could result in a significant and possibly costly loss of reputation. These attacks should not be a problem for a company which operates exclusively within their national boundaries. These companies can employ whatever cryptography they require for the full implementation of their security policy. When a company's operations take it across national boundaries, the vexed question of export is raised again.

A possible solution to the whole question of the export of cryptography has been proposed in the last few years. Clearly with all indications being that electronic commerce will grow, it is important to allow companies to operate effectively across national boundaries. It has been suggested that governments will permit the export of stronger cryptography giving strong confidentiality provided the key has been lodged, or escrowed, with an appropriate authority.

Indeed, the US government has said it will grant export licences for 56-bit DES provided companies work towards having a suitable key escrow scheme in place within two years. Not surprisingly key escrow schemes have aroused a certain degree of controversy.

5.2 Key escrow

Over the last few years there has been a significant increase in the number of people with access to, and using strong cryptosystems for confidentiality. This disturbs law enforcement agencies and national security agencies who have traditionally been able to eavesdrop on conversations in pursuit of their legitimate activities. However, these agencies also recognise the equally legitimate needs of individuals and businesses to be able to conduct their lawful business with some degree of privacy and security. In order to balance these two conflicting needs it was proposed, firstly in the US, to use escrowed key cryptography. Key escrow basically implies that a third party, possibly a governmental organisation keeps all, or part, of your private key until such time as they are presented with a warrant or other legal empowerment which grants the law enforcement agencies access to your key.

The system first introduced in the US was known as Clipper and consisted of a hardware encryption device with hardwired keys that were copied, split into two parts and lodged with government bodies [20]. This system was apparently introduced with very little consultation with potential users and, not surprisingly, there was an enormous resistance both to the general concept of escrow and the specific implementation of Clipper. One of the features of the Clipper chip was that it contained an algorithm developed by the NSA known as Skipjack which was claimed to offer strong confidentiality. The details of Skipjack were not released. It is fair to say that the take up of Clipper in the US has been minimal and many observers now believe escrow is off the menu for some time. The Clipper debacle in the US sparked a huge, vociferous and very public debate about the question of escrow and legal interception of communications. In the UK the debate has been somewhat muted despite there being very real proposals for key escrow under discussion. Again the system in the UK aims to balance the needs of individuals for confidentiality with the requirements of the law enforcement agencies to intercept communications, if necessary. One of the problems of the approach adopted by the UK is that it utilises a TTP infrastructure. This is not a problem in itself for as we shall see the UK proposals are a nice attempt to strike the right balance consistent with likely emerging technologies. However, the problem has been that some people have now erroneously identified the concept of a TTP infrastructure with that of key escrow. A TTP infrastructure is necessary to provide the trust in an

electronic business environment. The services supplied by the TTPs on this network do not have to include escrow.

We shall discuss the UK proposals in some detail in section 5.3. Firstly, however, it is useful to understand some points about the issue of key escrow in general. In the UK at least the 1985 Interception of Telecommunications Act enshrines in law the right of the security services and law enforcement agencies to eavesdrop on any communication upon production of the appropriate warrant. UK telcos operate within the law and must comply with this act. The ability to eavesdrop on encrypted communications through the use of escrow techniques does not give these agencies any fundamental new powers. It merely preserves an already existing capability. Balancing the freedoms required by a civilised society with the necessary restrictions required to guarantee those freedoms is a difficult task.

Escrow is not a substitute for proper back-up procedures. Sensitive data, including key material, should be regularly backed-up. Should the secret key become lost, key recovery is simply a matter of obtaining the back-up copy. The back-up procedures are a matter of internal policy and key escrow should not be seen as a replacement for those procedures. Key escrow techniques can, however, be used as a means of implementing a key recovery policy although other techniques can also be used. Escrow is really about the interception of communications not about recovery of keys.

5.3 The UK key escrow proposal

The escrow scheme favoured by the UK government for use on its own networks is based upon a system invented by the team based at Royal Holloway, London [21]. This scheme is built around a TTP infrastructure and was designed to meet a set of reasonable criteria. Given that it is likely that a TTP infrastructure will emerge in support of electronic commerce it would seem sensible to design a key escrow scheme consistent with this future framework. At present there are no proposals to outlaw the use of any form of cryptography in the UK within its borders. However, it is likely that TTPs providing confidentiality services will not be granted an operating licence unless they also implement some suitable escrow mechanism.

The objectives, or design criteria, that the Royal Holloway (RH) scheme [21] was engineered to satisfy are that:

- use of the scheme should be beneficial for the user,
- the scheme should allow both national and international operation,

- details of the scheme should be made public,
- it should be based upon well-known technique,
- all forms of electronic communication should be supported,
- the scheme should be compatible with existing legislation on the use, export and sale of cryptographic mechanisms and with warranted interception,
- warranted access must be provided to both incoming and outgoing communication,
- the scheme should be capable of supporting a variety of encryption algorithms in both hardware and software,
- the agency with warranted access should not be able to fabricate false evidence,
- if required users should be able to update keys according to their own internal security policies,
- abuse by either side should be detectable by the other,
- users should only have to communicate with their own TTP,
- on-line communication between TTPs should not be required.

The scheme in broad terms is an authenticated Diffie-Hellman key exchange where the TTPs provide the authentication and generate some of the key material. We shall follow the presentation of Jeffries, Mitchell and Walker closely, although for more detail the reader is referred to the original [21]. As with the original paper we shall, for convenience, only consider one-way communication where Alice wishes to send an encrypted e-mail to Bob.

We shall consider the situation where Alice and Bob are clients of different TTPs labelled TTP_A and TTP_B , respectively. Before Alice and Bob can communicate, their respective TTPs must agree on some parameters. We shall assume that the appropriate authenticity and confidentiality mechanisms are in place between the TTPs and in particular we assume they share a secret key, K_{AB} . Alice and Bob will use the Diffie-Hellman key exchange system to exchange a key for use in a secret-key cryptosystem. The actual secret-key encryption algorithm is largely irrelevant for our purposes here.

The TTPs agree between them a common modulus g and a large prime p for use with the Diffie-Hellman key exchange mechanism. TTP_A may establish a different value for g and p with another TTP. They also agree on a

deterministic key generating function f which takes as input the client's name and the shared secret key K_{AB} . The output from this function is an integer which becomes that client's private receive key. Thus Bob has a private receive key, β , given by:

$$\beta = f(\text{Bob}, K_{AB}) \quad \text{.....(28)}$$

From this Bob's public receive key can be calculated as:

$$g^\beta \bmod p = g^{f(\text{Bob}, K_{AB})} \bmod p \quad \text{..... (29)}$$

This key becomes Bob's public key for secure communication between Bob and any of TTP_A 's clients. Similarly Alice must be equipped with a private send key, α . This can either be generated by her TTP or it could be generated by Alice herself. Either way her public send key must be certified by TTP_A and Alice should possess the appropriate certificate. Alice's public send key is calculated in the same way as Bob's public receive key and is $g^\alpha \bmod p$. Alice must also have access to Bob's public receive key. She can obtain the certified key from her TTP.

Alice is now in possession of the following:

- her own private send key,
- a certificate for her public send key signed by her own TTP,
- the public receive key of Bob certified by her TTP,
- the parameters g and p .

Alice is now in a position to send Bob an encrypted message. She sends Bob the following:

- the message encrypted using the session key $g^{\alpha\beta} \bmod p$,
- her own signed public send key $g^\alpha \bmod p$,
- the signed public key of Bob $g^\beta \bmod p$.

Upon receipt of this information Bob uses Alice's copy of his public receive key to find the corresponding private receive key, β (note that there will be a different receive key pair for each TTP that has an agreement with Bob's TTP). Bob can now easily generate the secret session key by taking Alice's signed send key and raising it to the power $\beta \bmod p$.

Warranted interception can now be achieved easily. The intercepting authority retrieves the private key of Alice (or Bob) from the TTP within its jurisdiction. From this the

corresponding secret session key can be calculated from the public key of Bob (Alice). Notice that, there is no need for the intercepting authority to obtain the keys of private keys of both users.

This escrow scheme has been constructed to satisfy the design requirements and provides a neat solution to escrow on a TTP infrastructure. There are other features of the scheme which we have not discussed but which are examined in more detail in Jeffries, Mitchell and Walker [21].

6. Conclusions

We have looked at some of the basic technologies that will underpin the security of a future global business environment. Strong cryptographic techniques for both authentication and confidentiality will allow us to have confidence in that environment. The design of good cryptosystems is something of a black art but the overall principles are understood. With recent advances in computing power the key size required to achieve a particular security level has increased. 40-bit secret-key cryptography no longer provides adequate security for systems carrying valuable data. The export controls on cryptography guarantee that exported commercial, off-the-shelf products use cryptography that is no stronger than a 40 bit secret-key system. The use of escrowed cryptography has meant that some governments are prepared to relax the export controls provided the keys are escrowed with them before export. This should be borne in mind when assessing products claiming to offer the same security as a 128-bit secret-key system. If you do not mind the exporting country's government holding most of the key, then these products probably offer good security.

Secret-key and public-key systems are fundamentally different beasts. Their security derives from different properties. However, both are important technologies for securing the networks of the future. In particular, the use of public-key techniques allows us to transfer the trust away from the network infrastructure and on to the data itself. Trusted data carries with it its own integrity and can be stored anywhere. This trust is supplied by an infrastructure of trusted third parties. It is this infrastructure that we believe to be necessary for the successful introduction of electronic commerce on a wide scale.

Acknowledgements

I would like to thank my many colleagues, for numerous discussions which have clarified my understanding. I have also based some of the exposition of this review on the notes from the excellent Advanced Cryptography Course

run by Professor J Massey (one of the co-inventors of the IDEA algorithm) each year in Switzerland.

References

- 1 Beker M and Piper F: 'Cipher systems', Northwood, London (1982).
- 2 National Bureau of Standards, NBS FIPS PUB 46: 'Data encryption standard', US Department of Commerce (January 1977).
- 3 Biham E and Shamir A: 'Differential cryptanalysis of DES-like Cryptosystems', *Journal of Cryptology*, 4, No 3 (1991).
- 4 Schneier B: 'Applied cryptography', 2nd edition, Wiley, New York (1996).
- 5 Stinson D R: 'Cryptography, theory and practice', CRC, Boca Raton (1995).
- 6 Menezes A J, van Oorschot P C and Vanstone S A: 'Handbook of applied cryptography', CRC, Boca Raton (1997).
- 7 Merkle R C and Hellman M E: 'On the security of multiple encryption', *Communications of the ACM*, 24, p 465 (1981).
- 8 Campbell K W and Wiener M J: 'DES is not a group', in: 'Advances in Cryptology — Proceedings of CRYPTO '92', Springer-Verlag, Berlin, pp 512-520 (1992).
- 9 ANSI X9.17 (Revised): 'American national standard for financial institution key management (wholesale)', American Bankers Association (1985).
- 10 Rueppel R A: 'Analysis and design of stream ciphers', Springer-Verlag, Berlin (1986).
- 11 Diffie W and Hellman M E: 'New directions in cryptography', *IEEE Transactions on information theory*, IT-22, p 644 (1976).
- 12 Rivest R L, Shamir A and Adleman LM: 'A method for obtaining digital signatures and public key cryptosystems', *Communications of the ACM*, 21, p 120 (1978).
- 13 Koblitz N: 'A Course in number theory and cryptography', Springer-Verlag, Berlin (1994).
- 14 Diffie W, van Oorschot P C and Wiener M J: 'Authenticated key exchanges', *Designs, Codes and Cryptography*, 2, p 107 (1992).
- 15 Zimmerman P R: 'The official PGP users guide', MIT Press, Boston (1995).
- 16 'Proposed federal information processing standard for secure hash standard', *Federal Register*, 57, No 21, p 3747 (January 1992).

- 17 National Institute of Standards and Technology, NIST FIPS PUB 186, 'Digital signature standard', US Department of Commerce (1994).
- 18 Skevington P and Hart T: 'Trusted third parties in electronic commerce', BT Technol J, 15, No 2, pp 39 — 44 (April 1997).
- 19 <ftp://ftp.research.alt.com/dist/mab/keylength.ps>
- 20 National Institute of Standards and Technology, NIST FIPS PUB 185: 'Escrowed encryption standard', US Department of Commerce (1994).
- 21 Jeffries N, Mitchell C and Walker M: 'Trusted third party based key management allowing warranted interception', Proceedings of the Public Key Infrastructure Workshop, Washington, NIST (1995).



Simon Pheonix graduated from York University in 1986. He joined BT in 1989 after having completed his PhD in theoretical quantum optics at Imperial College London. His research work at BT has involved fundamental investigations into the quantum nature of light and its application to telecommunications. After a period developing several new quantum cryptographic techniques and concepts he moved into the security futures team at BT Laboratories where his interests now include commercial applications of cryptographic techniques and trusted third party services.